

PWNING BLUETOOTH DEVICES IN UNEXPECTED WAYS

VIRTUALABS | LEHACK 2026



WHO AM I?



Damien Cauquil

- Security Engineer at Quarkslab
- Team Cryptobedded
- Bluetooth Low Energy addict since 2016



INTRODUCTION

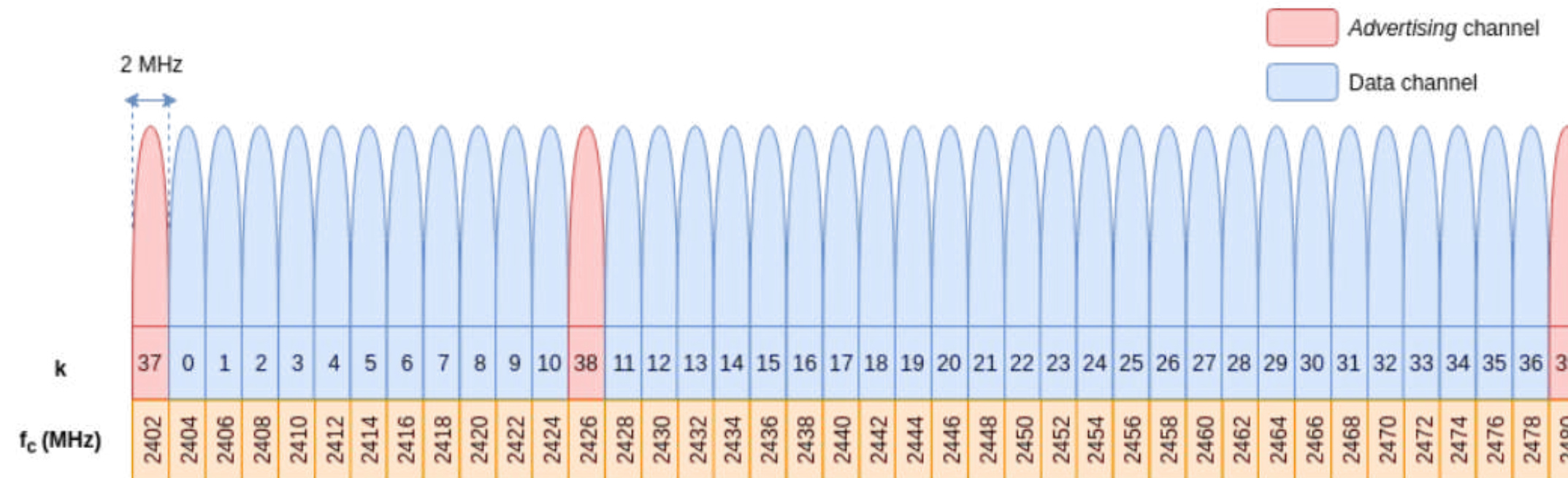
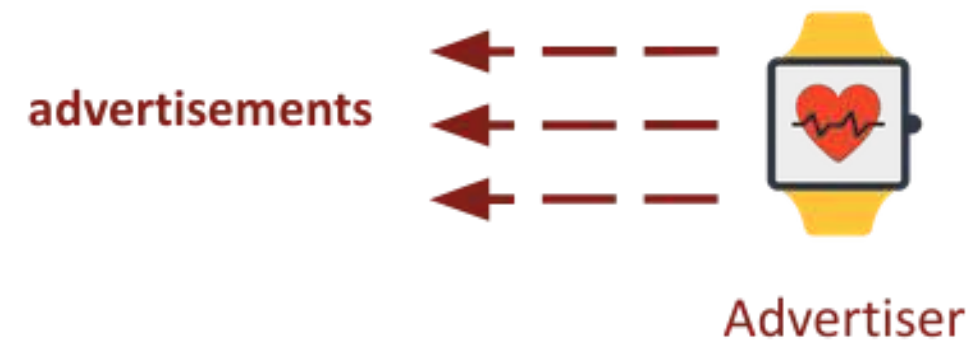
Bluetooth Low Energy (BLE) explained, short short version



BLE INTRODUCES LOW POWER

- One goal: save **power**!
- Exchange data at **regular interval** (7.5ms to 4s)
- Use **frequency hopping**
- **Re-use components** used in Bluetooth Classic

ADVERTISING PDUS (BROADCAST)





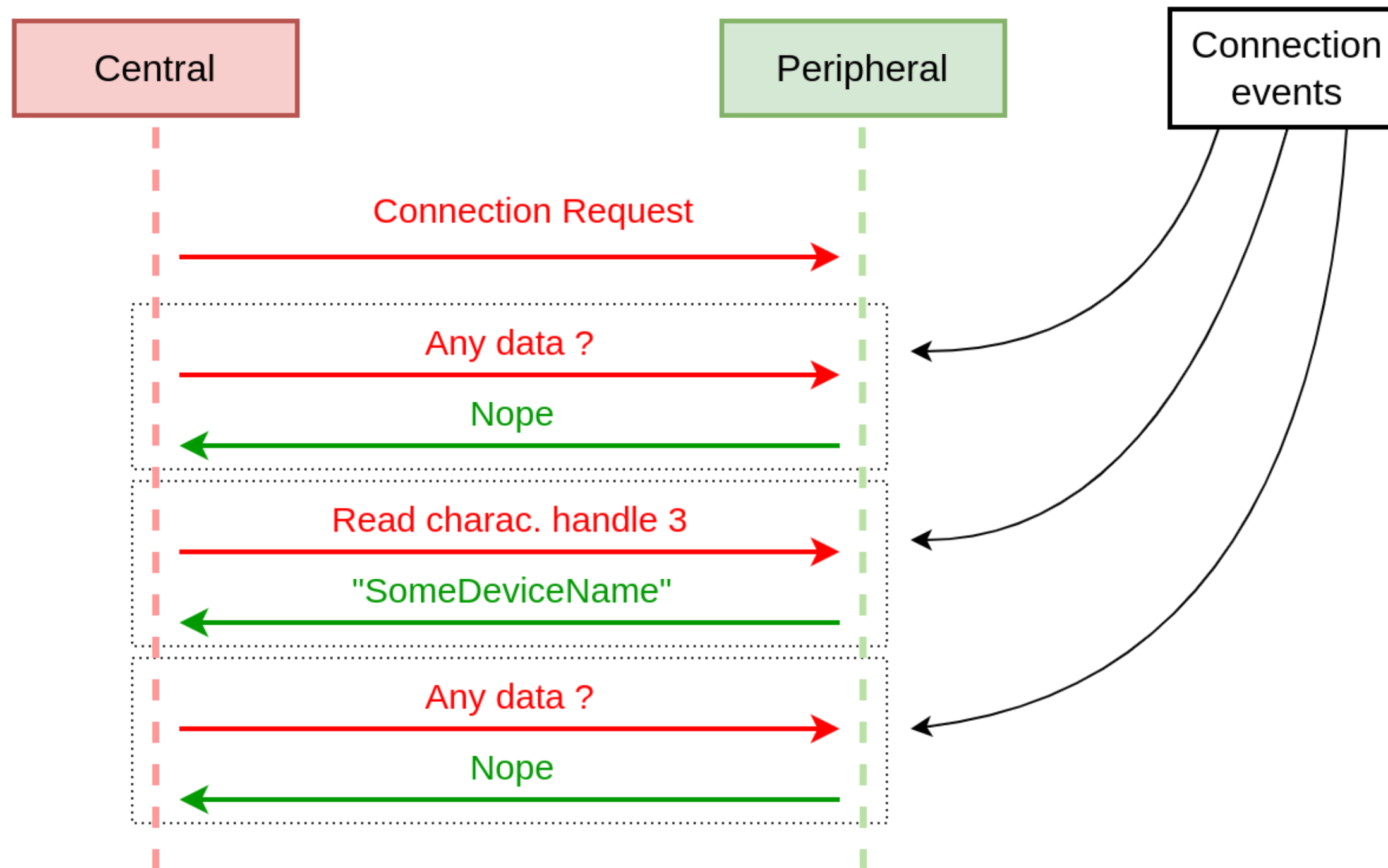
ADVERTISING DATA RECORDS (TLV)

Flags

Complete Name

02 06 01	07 09 6C 65 48 41 43 4B
----------	-------------------------

CONNECTING TO A BLE DEVICE





PROTOCOL STACK

GAP	G eneric A tttribute P rofile
GATT	G eneric ATT tribute P rotocol
ATT	ATT tribute P rotocol
L2CAP	Handles fragmentation and reassembly
Link-Layer	Exchange data over the air

EXAMPLE PDU

LL

L2CAP

ATT

02 07

03 00 04 00

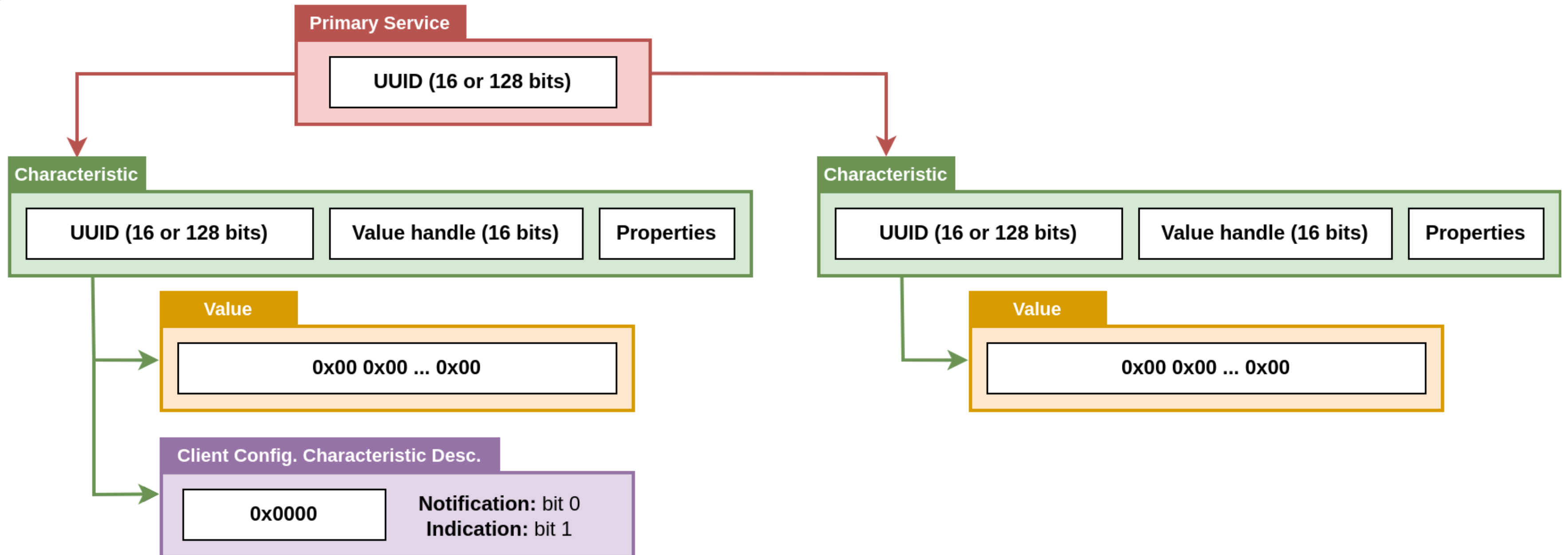
0A 03 00



ATTRIBUTE PROTOCOL

- Each attribute has a **type UUID** and a **16-bit handle**
- Defines some key attributes: **Primary service, Characteristic, ...**
- Defines a set of **operations** and **procedures**

ATTRIBUTE DATABASE





HOW IS DATA EXCHANGED?

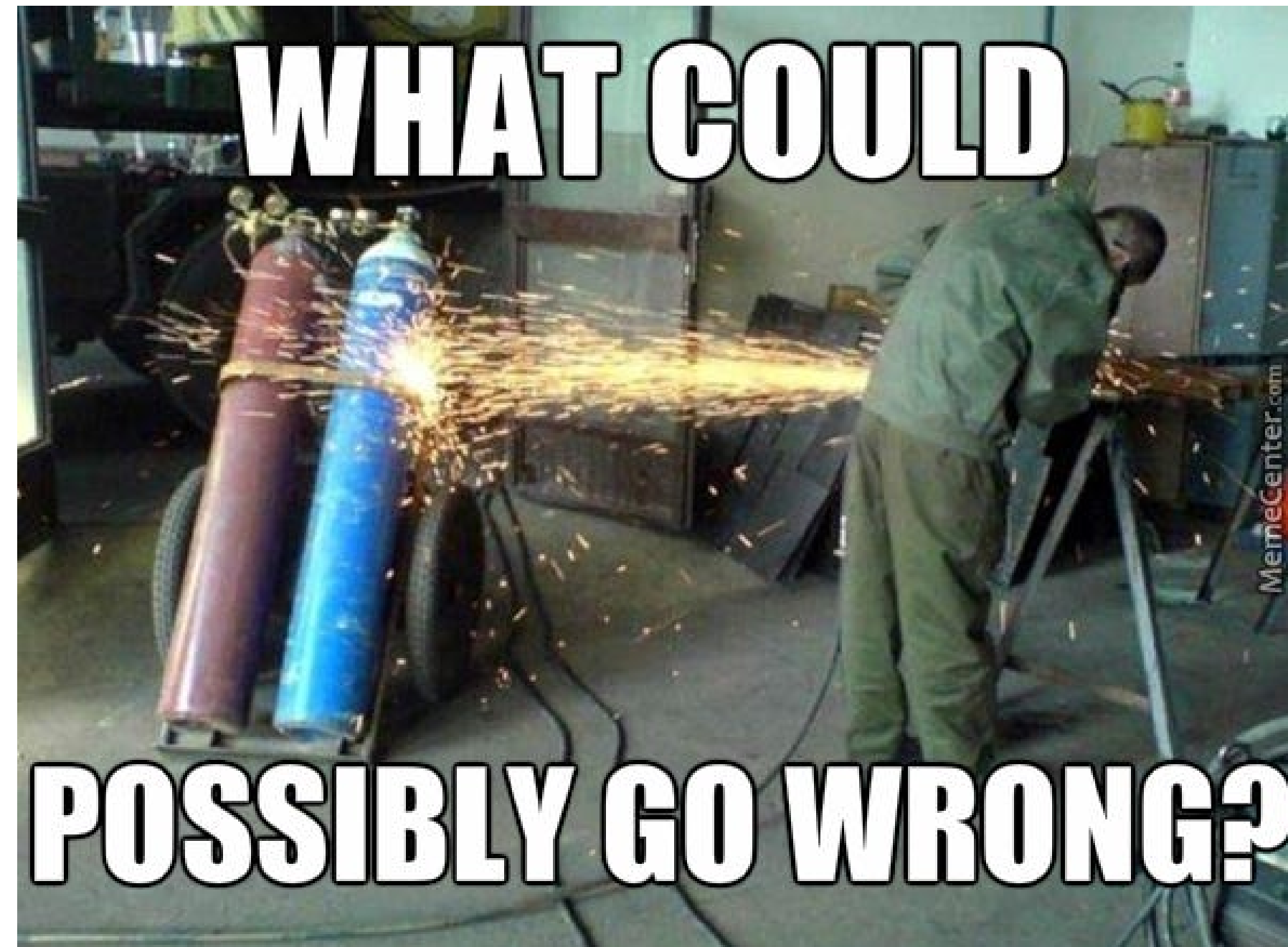
- Data is sent by **writing into a characteristic's value**
- Remote device answers by **updating a characteristic's value**
- **Notifications** are used to receive the new value when updated



SECURITY MECHANISMS

- Address randomization for **privacy**
- **Secure Connections** starting from version 4.2
- More **robust channel hopping** algorithm* (CSA #2)

(*) well, not really a security feature but it makes sniffing way harder 😅





10+ YEARS OF VULNERABILITIES...

- **Implementation issues:** RCE, DoS, ...
- **Design issues:** security downgrade, MitM, jamming
- **Crypto issues:** weak encryption, key entropy/reuse, ...

... **AND MAYBE MORE!**





LET'S TALK ABOUT PRIVACY

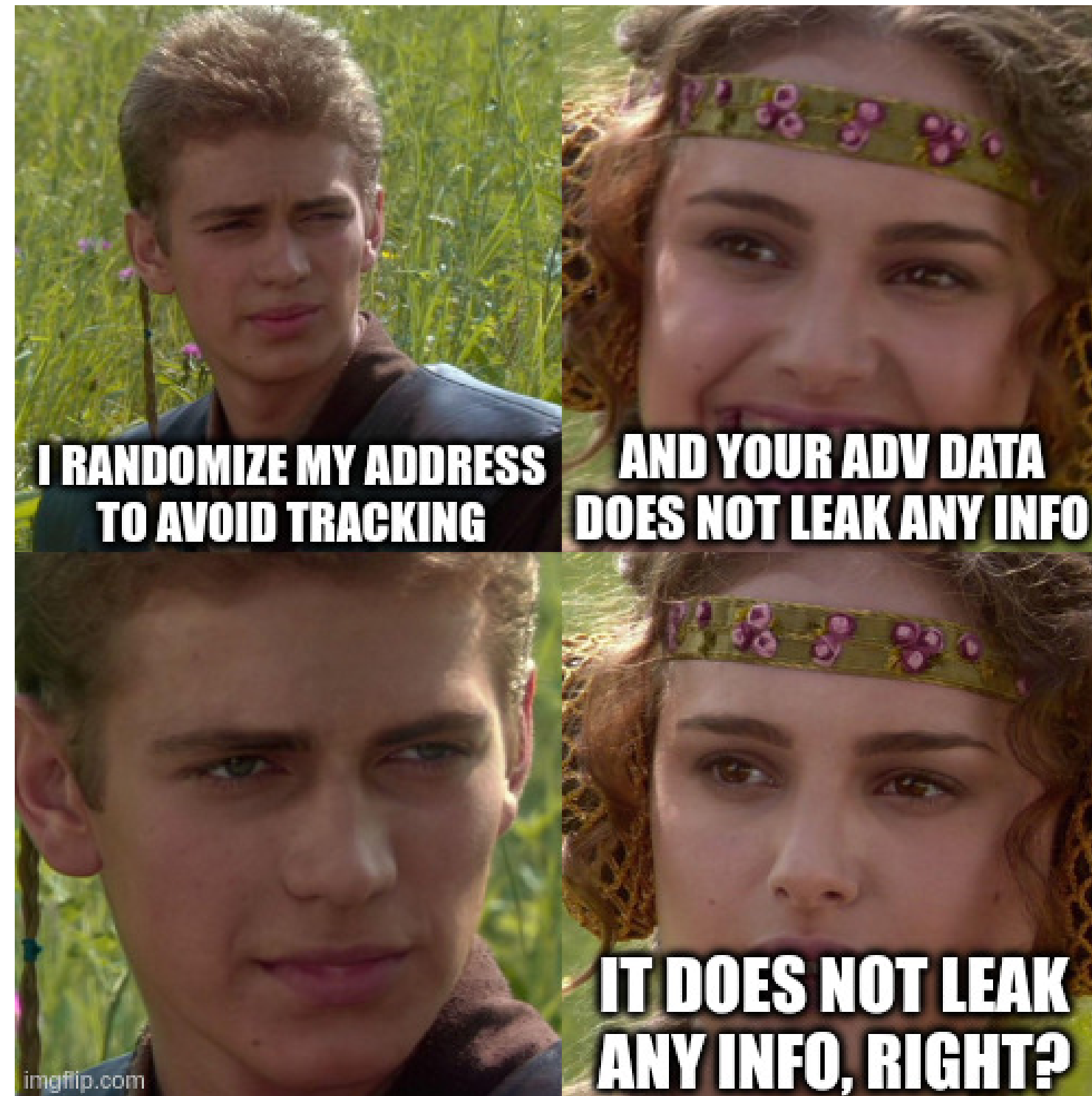




ADDRESS RANDOMIZATION

- **Non-Resolvable Private Address:**
randomized every X minutes to avoid tracking
- **Resolvable Private Address:**
Address contains a challenge/response (bonding required)

DEFEATING RANDOMIZATION



THE BOSE NC-700 CASE





NC-700 CHARACTERISTICS

- Bluetooth Classic and Bluetooth LE **at the same time**
- Smartphone connects over **Bluetooth Classic for audio**
- Can **control** the headset behavior **over Bluetooth LE**



MANUFACTURER DATA ADVERTISING RECORD

Header	Status	Unknown data
0C FF	51 12	28 a4 1e d5 2f 2a 88 7f 89



CUSTOM RESOLVABLE PRIVATE ADDRESS

- Bose created its **own Resolvable Private Address** mechanism
- Hardware still uses Bluetooth's RPA feature as well
- This weird mix **makes it weaker!**



Time elapsed	BD Addr.	Unique ID
+ 0s	47:e4:8d:3c:db:a4	a823f268c430
+ 234s	c8:7b:23:58:53:15	a823f268c430
+ 296s	c8:7b:23:58:53:15	28a41ed52f2a887f89



FINGERPRINTING



USING BLE CHARACTERISTICS

- Advertising data (ADV_IND + SCAN_RSP)
- Advertising interval
- LL_VERSION_IND

BASEBAND VENDOR & VERSION

2.4.2.13 *LL_VERSION_IND*

The format of the CtrData field is shown in [Figure 2.35](#).

CtrData		
Version (1 octet)	Company_Identifier (2 octets)	Subversion (2 octets)

BOSE NC-700 VERSION INFO

No.	Time	Source	Destin	Protocol	Length	Info
1	0.000000			LE LL	25	Control Opcode: LL_VERSION_IND
2	-1782197134.6...			LE LL	25	Control Opcode: LL_VERSION_IND
Frame 2: 25 bytes on wire (200 bits), 25 bytes captured on interface (200 bits) on Bluetooth						
Bluetooth Low Energy RF Info						
Bluetooth Low Energy Link Layer						
Access Address: 0x23a3d487						
Data Header						
Control Opcode: LL_VERSION_IND (0x0c)						
Version Number: 5.0 (0x09)						
Company Id: Qualcomm Technologies International						
Subversion Number: 0x38ce						
CRC: 0x93dcbf						
			0000	1c 00 80 00 00 00 00 00	83 0d 87 d4 a3 23 0b 06	...
			0010	0c 09 0a 00 ce 38	c9 3b fd	...



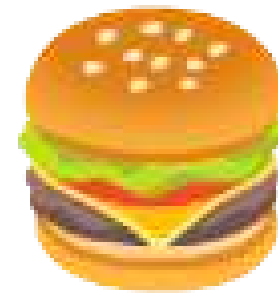
IMAGINE, AUTOPWN BLE DEVICES...

- Finding known vulnerabilities **based on stack fingerprint**
- **Automated exploitation** & compromise
- **Steal** data, **hack** everyting, cause **total chaos!**

(Well, it does not exist ... yet)



ABUSING BLUETOOTH LE STACKS





ATTRIBUTES DATABASE (GATT)

Handle	Type UUID	Value (Hex.)	Properties
1	0x2800	00 18	Read
2	0x2803	02 03 00 00 2A	Read
3	0x2A00	00 00 00 00 00 00	Read/Write



ATTRIBUTE PROPERTIES

- **ATT-level:** defines access rights for the attribute value
- **GATT-level:** defines access rights for a characteristic's value

CHECKING PERMISSIONS IS HARD

(venv) `virtualabs@virtubox:~$` 

▶ 0:00 / 0:52



WIRELESS DEFACING



hacked

E2:DB:31:E2:C9:ED

NOT BONDED

▲ -102 dBm ↔ 50 ms

CONNECT



Device type: LE only

Advertising type: Legacy

Flags: GeneralDiscoverable, BrEdrNotSupported

Complete list of 16-bit Service UUIDs: 0x180D

Complete Local Name: hacked

CLONE

RAW

MORE

DID I TELL YOU CHECKING PERMISSIONS WAS HARD?

wble-central>

I

▶ 0:00 / 0:52



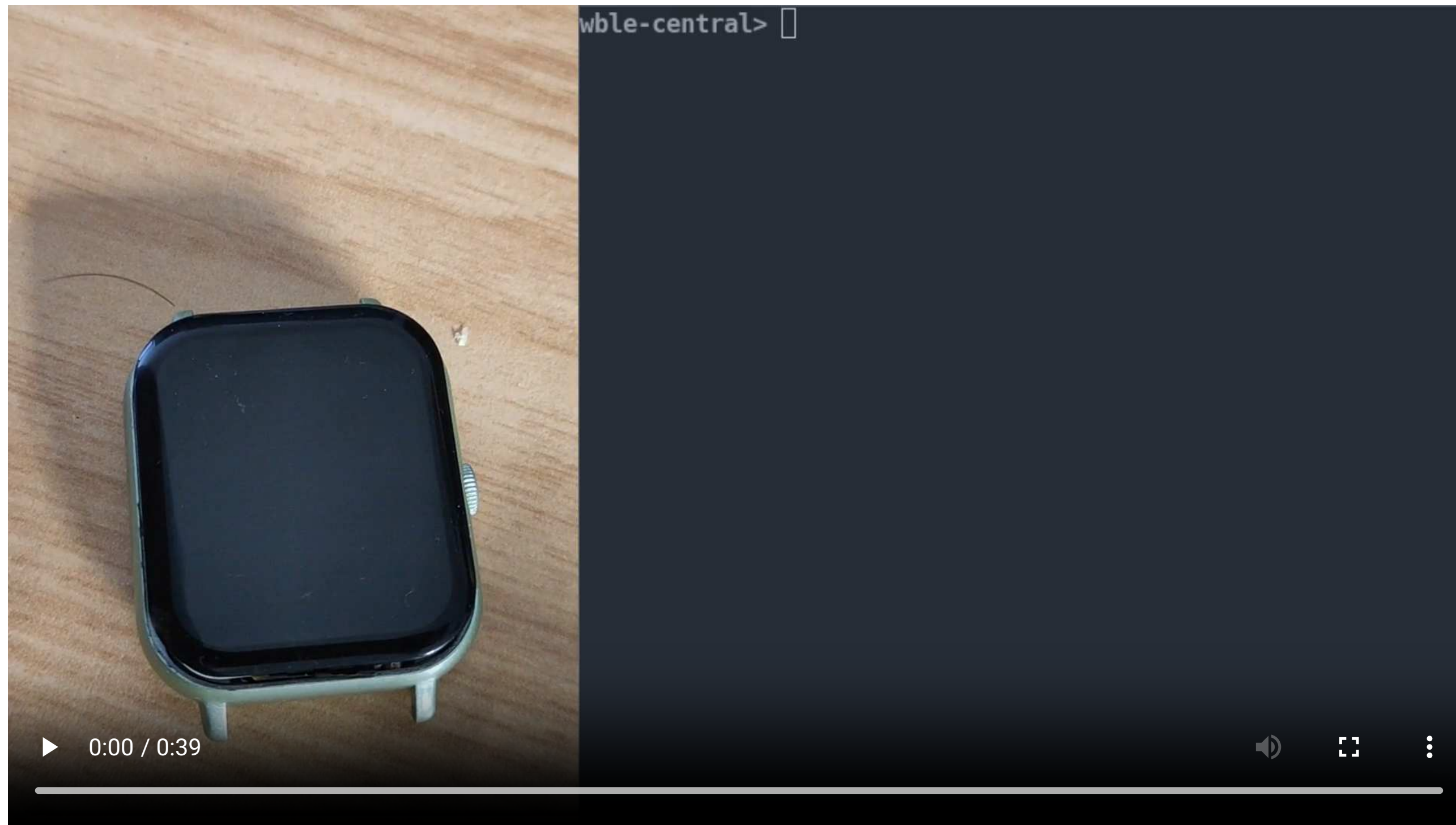
LIKE, REALLY HARD... 🤨

wble-central>



▶ 0:00 / 0:41 🔊 🗐 ⋮

SENDING UNEXPECTED PDUS

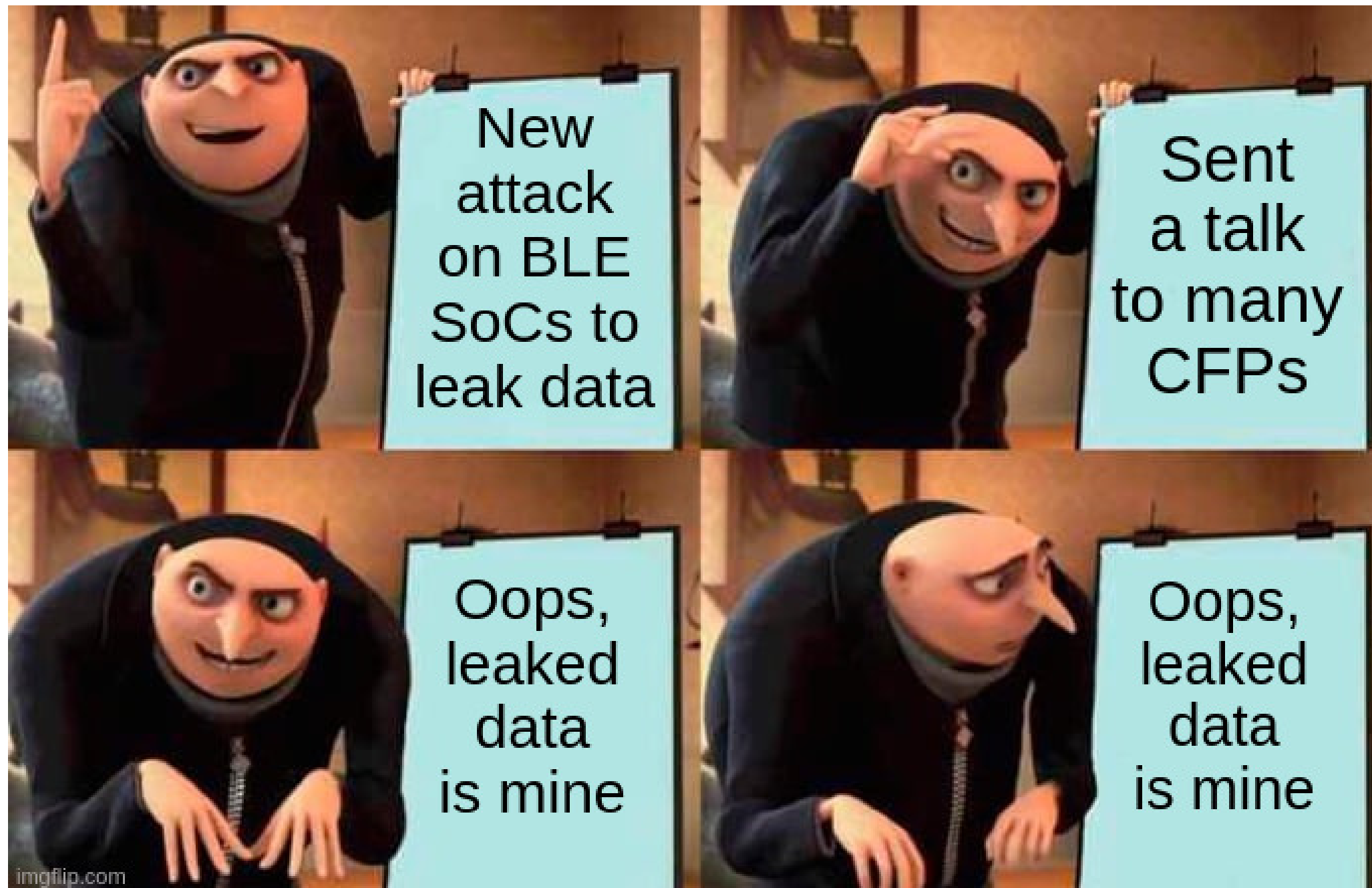




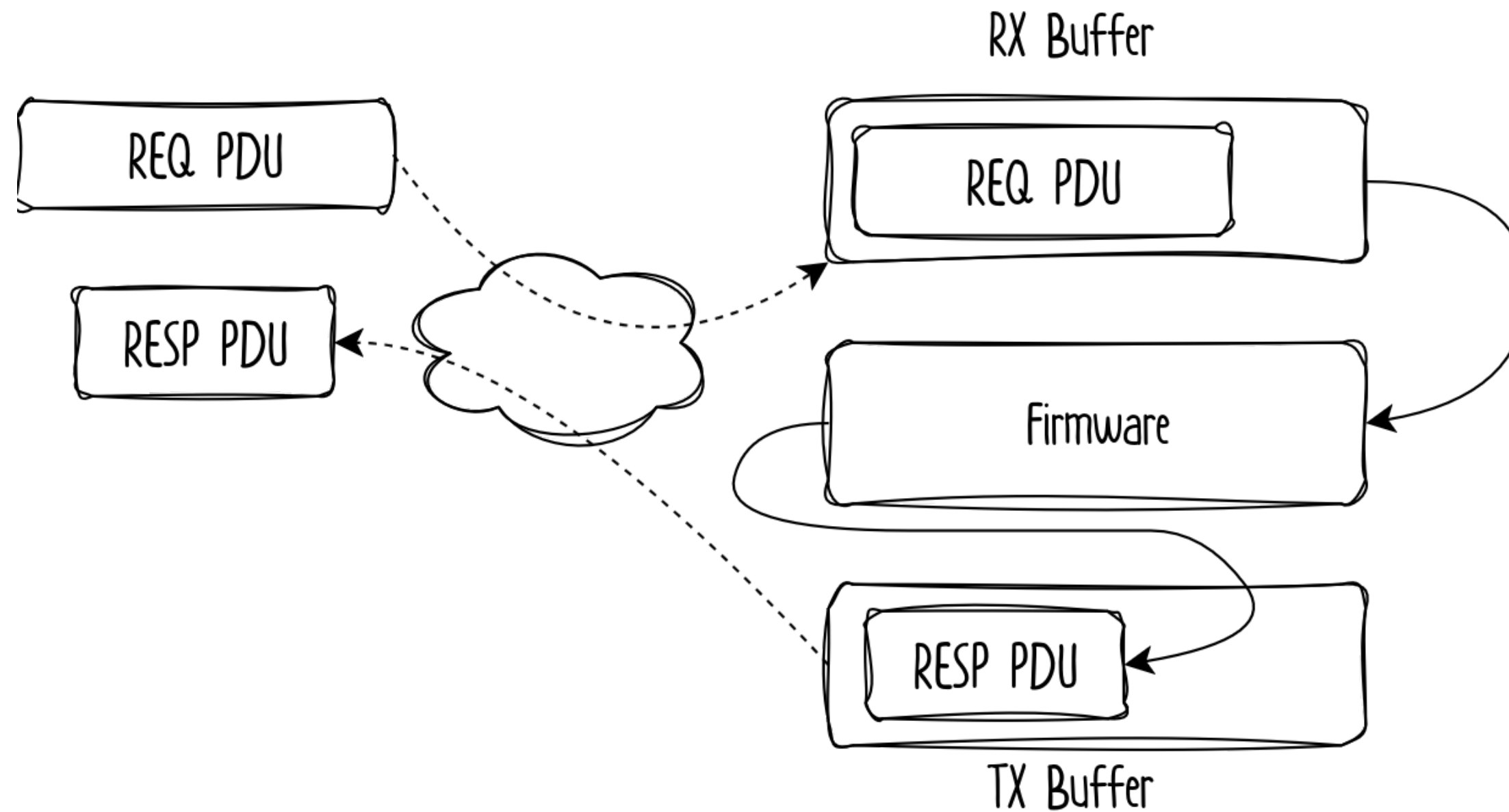
STEALING DATA FROM SYSTEM-ON-CHIPS

OR HOW I F*CKED UP MY CFP SUBMISSION 🙄

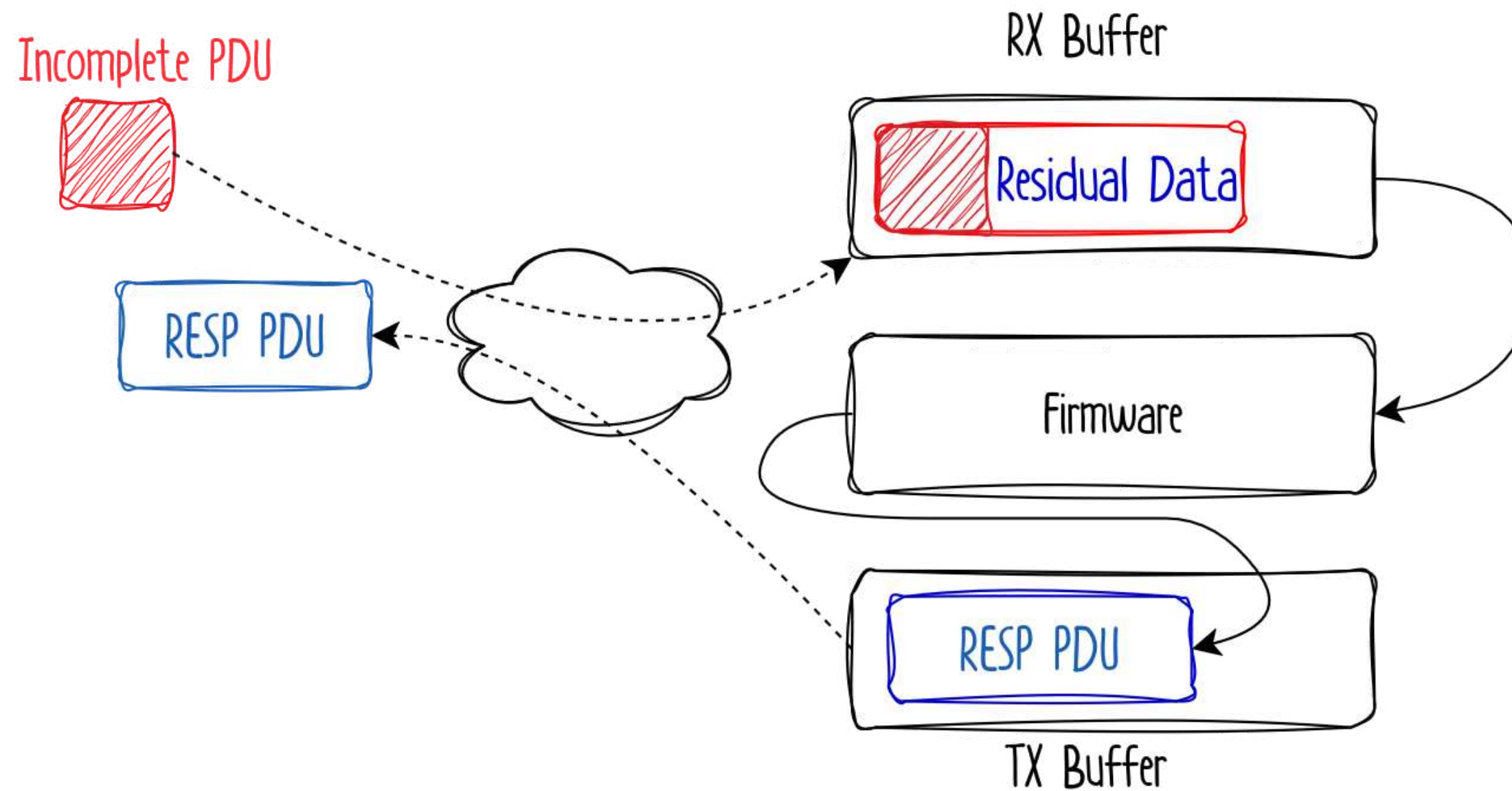
A FEW MONTHS AGO



BLUETOOTH PDU RX PROCESSING



MESSING WITH HEADER TO LEAK RESIDUAL DATA





THE PLAN

- Sending a **PrepareWrite Request** with bad length field
- Device **echoes the supplied data** (residual data!)



EXPLOITING THIS FLAW

- Successfully recovered data from **PeprareWrite Responses!**
- Did not notice something was off at this time...

OH CRAP, I LEAKED MY OWN DATA 🙈

LL

L2CAP

ATT

02 80

7C 00 04 00

16 03 00 00 00

Length: 128 ATT len: 124 Prepare Write Req

System-on-chips use the **length field**
to figure out the LL payload's size...

I'M A FRAUD.





WAIT A MINUTE...

SOMETHING'S WEIRD 🤔

	LL	L2CAP	ATT
TX:	02 05	<u>01 00</u> 04 00	0A
RX:	02 06	02 00 04 00	0B 64

Device answers to a **ATT_READ_REQ** that has no handle!



OMG, I HAVE A NEW PLAN!

- We can **leak** data by **abusing weak ATT length checks** 🎉
- **ATT** provides **all the primitives** we need!



LEAKING THE FIRST 2 BYTES

- We send an **ATT_READ_REQ** with no handle
- If the **next 2 bytes match a handle**, attribute's value is returned
- **If not**, an ATT_ERROR_RSP containing **the handle value is returned!**

LEAKING 2 MORE BYTES

3.4.3.1 ATT_FIND_INFORMATION_REQ

The ATT_FIND_INFORMATION_REQ PDU is used to obtain the mapping of attribute handles with their associated types. This allows a client to discover the list of attributes and their types on a server.

Parameter	Size (octets)	Description
Attribute Opcode	1	0x04 = ATT_FIND_INFORMATION_REQ
Starting Handle	2	First requested handle number
Ending Handle	2	Last requested handle number

Table 3.7: Format of ATT_FIND_INFORMATION_REQ PDU



THAT'S A 4-BYTE MEMORY LEAK!

- Exploits an **invalid ATT length check**
- Combines two ATT requests to **leak 4 bytes of residual data**

PROOF OF CONCEPT

(venv) **virtualabs@virtubox**:~\$

▶ 0:00 / 0:13





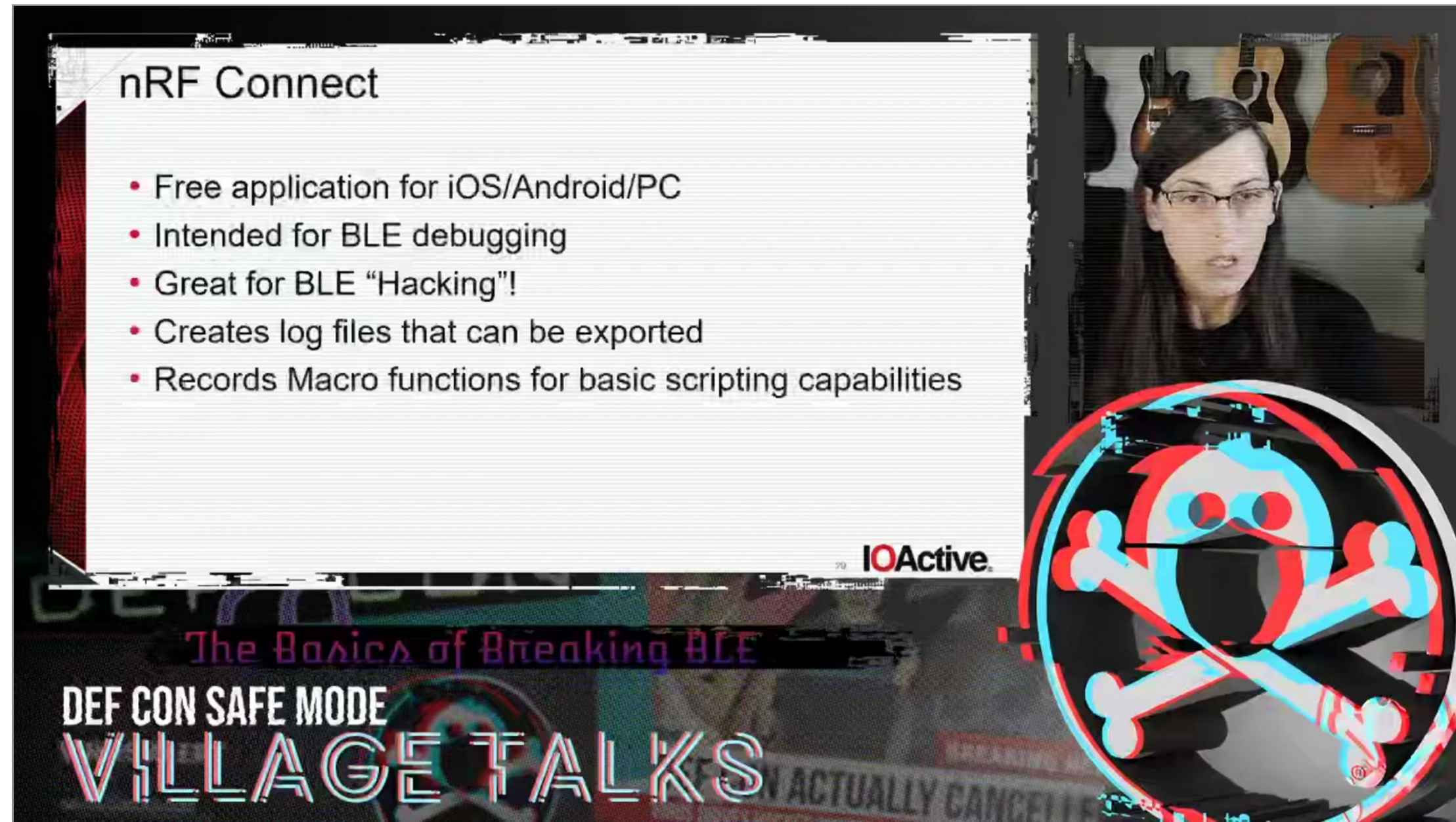
Vendor	Subversion	Vulnerable?
Bluetrum	0x0003	
JieLi	-	
Realtek	0x8773	
Qualcomm	0x38ce	
Cypress	0x2310	



HACKING HACKERS



WHAT COULD POSSIBLY GO WRONG?



The video frame displays a presentation slide titled "nRF Connect" with the following bullet points:

- Free application for iOS/Android/PC
- Intended for BLE debugging
- Great for BLE "Hacking"!
- Creates log files that can be exported
- Records Macro functions for basic scripting capabilities

The IOActive logo is visible in the bottom right corner of the slide. To the right of the slide, a woman with glasses is visible, presumably the speaker. In the bottom right corner of the video frame, there is a large, stylized skull-and-crossbones logo with a red and blue 3D effect.

DEF CON SAFE MODE
VILLAGE TALKS

DEF CON Safe Mode Wireless Village - FreqyXin - The Basics Of Breaking BLE v3



BLE PERIPHERAL WITH A GATT CLIENT

- **Totally legit**, defined in Bluetooth Specification 🤔
- Yes, a **Peripheral can send requests** to a Central
- It's some kind of **reverse shell** I guess 😊

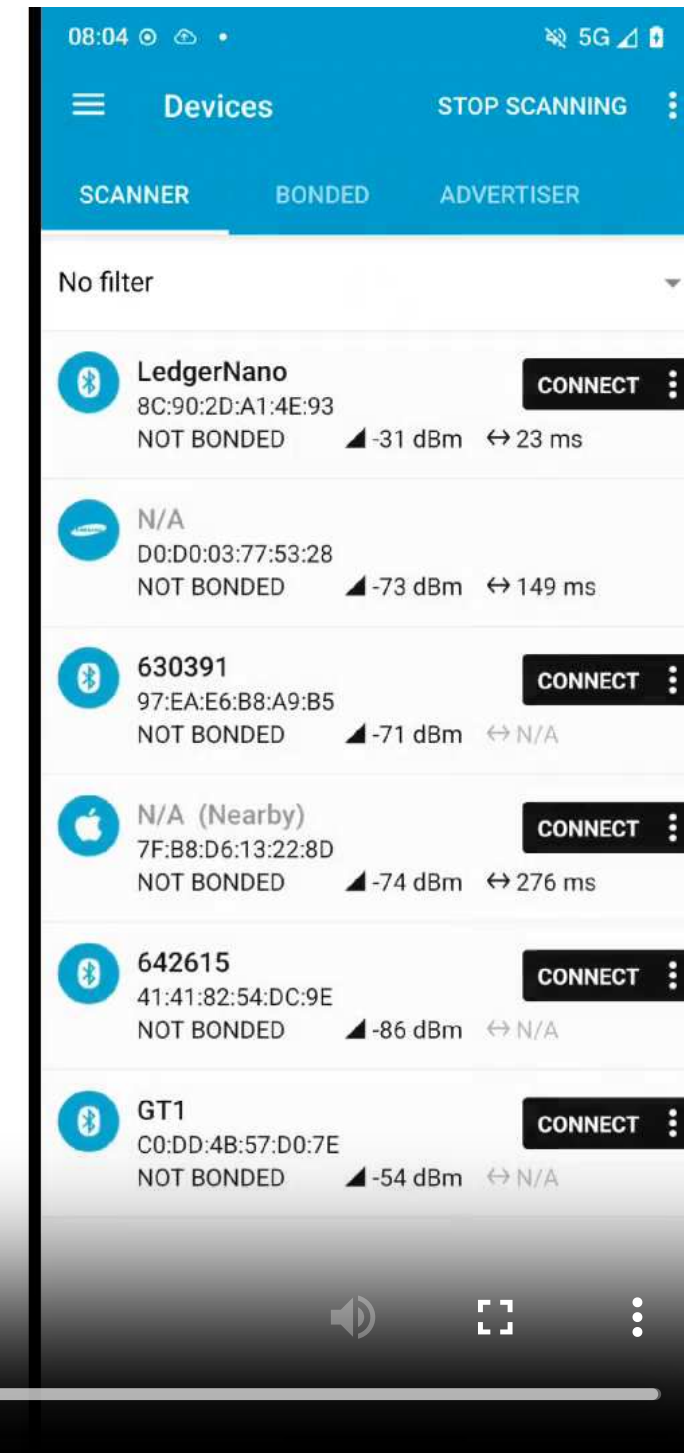


HACKERS WHINED DURING EC2

*We saw the 'BleCallsHome' Bluetooth device
but there is no characteristic marked as
writable... What are we supposed to do?*

REVERSE PWN

```
(venv) virtualabs@virtubox:~$ python3 reverse-fp.py hci1
```



0:00 / 0:21

PUTTING EVERYTHING TOGETHER

- Spawn a **juicy BLE Peripheral** device with a **GATT client** in it
- **Wait** for a hacker to connect to it
- **Fingerprint its BLE stack** and find the best exploit technique
- **Gain access** to the hacker's device 🎉



CONCLUSION



BLE IS NOT EASY

- App/firmware developers can **ruin security**
- Some BLE stacks **fail to correctly parse PDUs** (we're in 2026, guys...)
- BLE **Peripheral** role can have a **GATT client** and use it



HOW TO AVOID BEING HACKED?

- **Keep** firmwares and OS **up-to-date**
- **Don't connect to random devices** using your smartphone
- Don't use devices based on **cheap Chinese stacks** (for now)



THANKS ! WANT TO DIG DEEPER?



Workshop: "Hacking bluetooth devices with WHAD"

Mastodon: @virtualabs@mamot.fr